Lab Experiments 24-40

Machine Learning & Data Analysis: Python Code & Output

Experiment 24

KNN Classifier for Medical Condition Prediction

PYTHON

Code

```
# Q24: KNN Classifier for medical condition prediction
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("patients.csv")
X = df[["symptom1_fever", "symptom2_fatigue", "symptom3_pain", "symptom4_cough"]]
y = df["condition"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

k = int(input("Enter the value of k (number of neighbors): "))

model = KNeighborsClassifier(n_neighbors=k)
model.fit(X_train_scaled, y_train)

print(f"Model accuracy on test data: {model.score(X_test_scaled, y_test):.2f}")

print("\nEnter the new patient's symptom scores (0-10):")
fever = float(input("Fever severity: "))
fatigue = float(input("Fatigue level: "))
pain = float(input("Pain level: "))
cough = float(input("Cough severity: "))

new_patient = scaler.transform([[fever, fatigue, pain, cough]])
prediction = model.predict(new_patient)[0]

print("\nPrediction:", "Has the medical condition" if prediction == 1 else "Does not have the medical con
    dition")
```

Output

```
Enter the value of k (number of neighbors): Model accuracy on test data: 0.83

Enter the new patient's symptom scores (0-10):
Fever severity: Fatigue level: Pain level: Cough severity:
Prediction: Does not have the medical condition
```

Experiment 25

Decision Tree for Iris Flower Classification

PYTHON

Code

```
# Q25: Decision Tree for Iris Flower Classification
import warnings; warnings.filterwarnings("ignore")
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()
X, y = iris.data, iris.target

model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

print("Enter the new flower's measurements (cm):")
sepal_length = float(input("Sepal length: "))
sepal_width = float(input("Sepal width: "))
petal_length = float(input("Petal length: "))
petal_width = float(input("Petal width: "))

new_flower = [[sepal_length, sepal_width, petal_length, petal_width]]
prediction = model.predict(new_flower)[0]

print("\nPredicted species:", iris.target_names[prediction])
```

Output

```
Enter the new flower's measurements (cm):
Sepal length: Sepal width: Petal length: Petal width:
Predicted species: setosa
```

Experiment 26

Linear Regression for Housing Price Prediction

PYTHON

Code

```
# Q26: Linear Regression for Housing Price Prediction
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

df = pd.read_csv("housing.csv")
X = df[["area_sqft", "bedrooms", "location_score"]]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

print(f"Model R-squared on test data: {model.score(X_test, y_test):.3f}")

print("\nEnter the new house's features:")
area = float(input("Area (sqft): "))
bedrooms = int(input("Number of bedrooms: "))
location_score = int(input("Location score (1-10): "))

new_house = [[area, bedrooms, location_score]]
predicted_price = model.predict(new_house)[0]

print(f"\nPredicted price: {predicted_price:.2f}")
```

Output

```
Model R-squared on test data: 0.946

Enter the new house's features:
Area (sqft): Number of bedrooms: Location score (1-10):
Predicted price: 783726.72
```

Experiment 27

Logistic Regression for Customer Churn Prediction

PYTHON

Code

```
# Q27: Logistic Regression for Customer Churn Prediction
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("churn.csv")
X = df[["usage_minutes", "contract_duration_months", "support_calls"]]
y = df["churn"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression()
model.fit(X_train_scaled, y_train)

print(f"Model accuracy on test data: {model.score(X_test_scaled, y_test):.2f}")

print("\nEnter the new customer's details:")
usage = float(input("Usage minutes: "))
duration = int(input("Contract duration (months): "))
calls = int(input("Number of support calls: "))

new_customer = scaler.transform([[usage, duration, calls]])
prediction = model.predict(new_customer)[0]

print("\nPrediction:", "Customer will churn" if prediction == 1 else "Customer will not churn")
```

Output

```
Model accuracy on test data: 0.75

Enter the new customer's details:
Usage minutes: Contract duration (months): Number of support calls:
Prediction: Customer will not churn
```

Experiment 28

K-Means Clustering for Customer Segmentation

PYTHON

Code

```
# Q28: K-Means Clustering for Customer Segmentation
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("customers_shopping.csv")
X = df[["annual_spend", "purchase_frequency", "avg_basket_size"]]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = KMeans(n_clusters=4, n_init=10, random_state=42)
model.fit(X_scaled)

print("Enter the new customer's shopping features:")
spend = float(input("Annual spend: "))
frequency = float(input("Purchase frequency (times/year): "))
basket = float(input("Average basket size: "))

new_customer = scaler.transform([[spend, frequency, basket]])
segment = model.predict(new_customer)[0]

print(f"\nThis customer belongs to Segment {segment}")
print("Segment sizes:", pd.Series(model.labels_).value_counts().sort_index().to_dict())
```

Output

```
Enter the new customer's shopping features:
Annual spend: Purchase frequency (times/year): Average basket size:
This customer belongs to Segment 1
Segment sizes: {0: 35, 1: 43, 2: 35, 3: 37}
```

Experiment 29

Evaluation Metrics for Model Performance

PYTHON

Code

```
# Q29: Evaluation Metrics for Model Performance
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

df = pd.read_csv("churn.csv")
print("Available columns:", list(df.columns))

feature_input = input("Enter feature column names, comma-separated: ")
target_input = input("Enter the target column name: ")

features = [f.strip() for f in feature_input.split(",")]
target = target_input.strip()

X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)

print("\nModel Evaluation Metrics:")
print("Accuracy: ", round(accuracy_score(y_test, y_pred), 3))
print("Precision:", round(precision_score(y_test, y_pred), 3))
print("Recall:   ", round(recall_score(y_test, y_pred), 3))
print("F1-Score: ", round(f1_score(y_test, y_pred), 3))
```

Output

```
Available columns: ['usage_minutes', 'contract_duration_months', 'support_calls', 'churn']
Enter feature column names, comma-separated: Enter the target column name:
Model Evaluation Metrics:
Accuracy:  0.75
Precision: 0.714
Recall:    0.789
F1-Score:  0.75
```

Experiment 30

CART for Car Price Prediction

PYTHON

Code

```
# Q30: CART for Car Price Prediction
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
from sklearn.tree import DecisionTreeRegressor, export_text
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

df = pd.read_csv("cars_cart.csv")

le_brand = LabelEncoder()
le_engine = LabelEncoder()
df["brand_enc"] = le_brand.fit_transform(df["brand"])
df["engine_enc"] = le_engine.fit_transform(df["engine_type"])

X = df[["mileage", "age_years", "brand_enc", "engine_enc"]]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeRegressor(max_depth=4, random_state=42)
model.fit(X_train, y_train)

print(f"Model R-squared on test data: {model.score(X_test, y_test):.3f}")

print("\nEnter the new car's details:")
mileage = float(input("Mileage: "))
age = int(input("Age (years): "))
brand = input(f"Brand {list(le_brand.classes_)}: ")
engine = input(f"Engine type {list(le_engine.classes_)}: ")

brand_enc = le_brand.transform([brand])[0]
engine_enc = le_engine.transform([engine])[0]

new_car = [[mileage, age, brand_enc, engine_enc]]
```

```
predicted_price = model.predict(new_car)[0]

print(f"\nPredicted price: {predicted_price:.2f}")
print("\nDecision path:")
print(export_text(model, feature_names=["mileage", "age_years", "brand_enc", "engine_enc"]))
```

Output

```
Model R-squared on test data: 0.857

Enter the new car's details:
Mileage: Age (years): Brand ['BMW', 'Ford', 'Honda', 'Hyundai', 'Toyota']: Engine type ['Diesel', 'Electr
    ic', 'Hybrid', 'Petrol']:
Predicted price: 17688.53

Decision path:
|--- age_years <= 10.50
|   |--- mileage <= 32387.50
|   |   |--- mileage <= 29421.50
|   |   |   |--- age_years <= 3.50
|   |   |   |   |--- value: [23720.84]
|   |   |   |--- age_years >  3.50
|   |   |   |   |--- value: [21291.83]
|   |   |--- mileage >  29421.50
|   |   |   |--- age_years <= 0.50
|   |   |   |   |--- value: [28505.84]
|   |   |   |--- age_years >  0.50
|   |   |   |   |--- value: [27330.21]
|   |--- mileage >  32387.50
|   |   |--- age_years <= 5.50
|   |   |   |--- age_years <= 0.50
|   |   |   |   |--- value: [20843.61]
|   |   |   |--- age_years >  0.50
|   |   |   |   |--- value: [17688.53]
|   |   |--- age_years >  5.50
|   |   |   |--- mileage <= 111881.00
|   |   |   |   |--- value: [15961.00]
|   |   |   |--- mileage >  111881.00
|   |   |   |   |--- value: [13125.08]
|--- age_years >  10.50
|   |--- brand_enc <= 0.50
|   |   |--- mileage <= 127950.50
|   |   |   |--- mileage <= 38570.00
|   |   |   |   |--- value: [18860.45]
|   |   |   |--- mileage >  38570.00
|   |   |   |   |--- value: [15777.72]
|   |   |--- mileage >  127950.50
|   |   |   |--- mileage <= 147828.00
|   |   |   |   |--- value: [12781.07]
|   |   |   |--- mileage >  147828.00
|   |   |   |   |--- value: [11731.62]
|   |--- brand_enc >  0.50
|   |   |--- mileage <= 67316.50
|   |   |   |--- mileage <= 29775.00
|   |   |   |   |--- value: [14464.97]
|   |   |   |--- mileage >  29775.00
|   |   |   |   |--- value: [12059.62]
|   |   |--- mileage >  67316.50
|   |   |   |--- age_years <= 13.50
|   |   |   |   |--- value: [9578.10]
|   |   |   |--- age_years >  13.50
|   |   |   |   |--- value: [7368.68]
```

Experiment 31

Customer Segmentation Using Clustering

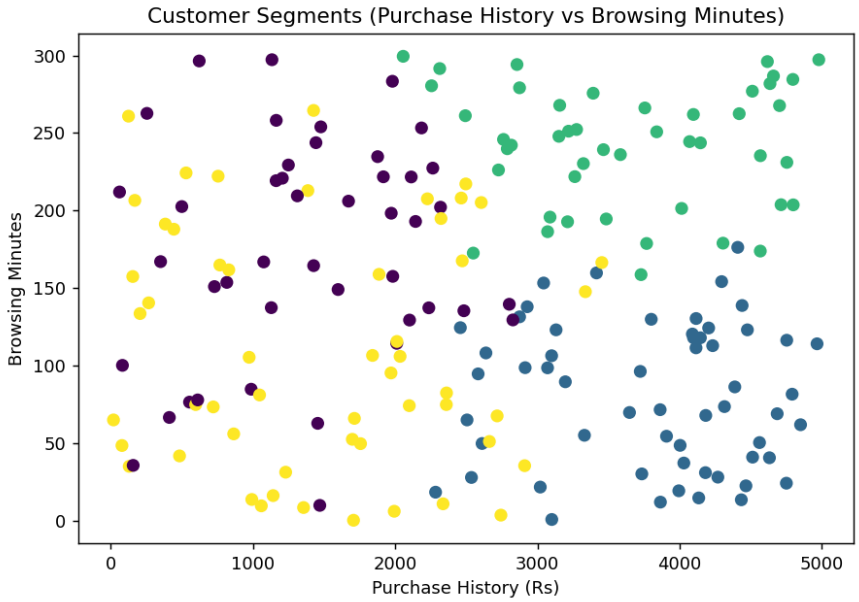
PYTHON

Code

```
# Q31: Customer Segmentation Using Clustering
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("ecommerce_customers.csv")
X = df[["purchase_history", "browsing_minutes", "age"]]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = KMeans(n_clusters=4, n_init=10, random_state=42)
df["segment"] = model.fit_predict(X_scaled)

print("Customer segment sizes:")
print(df["segment"].value_counts().sort_index())

print("\nSegment profile (mean values):")
print(df.groupby("segment")[["purchase_history", "browsing_minutes", "age"]].mean().round(2))

plt.figure(figsize=(7, 5))
plt.scatter(df["purchase_history"], df["browsing_minutes"], c=df["segment"], cmap="viridis", s=40)
plt.title("Customer Segments (Purchase History vs Browsing Minutes)")
plt.xlabel("Purchase History (Rs)")
plt.ylabel("Browsing Minutes")
plt.tight_layout()
plt.show()
```

Output

```
Customer segment sizes:
segment
0    44
1    57
2    46
3    53
Name: count, dtype: int64

Segment profile (mean values):
         purchase_history  browsing_minutes    age
segment
0                 1393.75            174.90  29.39
1                 3803.15             80.69  41.09
2                 3643.45            241.57  36.91
3                 1477.53            110.60  54.68
```



Experiment 32

Bivariate Analysis + Linear Regression for House Prices

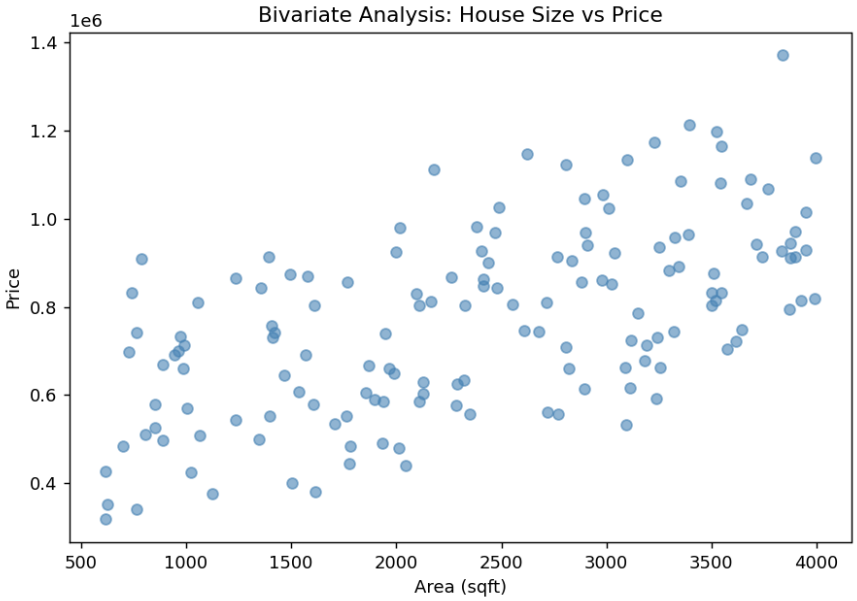
PYTHON

Code

```
# Q32: Bivariate Analysis + Linear Regression for House Prices
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error

df = pd.read_csv("housing.csv")

# Bivariate analysis: house size (area) vs price
plt.figure(figsize=(7, 5))
plt.scatter(df["area_sqft"], df["price"], alpha=0.6, color="steelblue")
plt.title("Bivariate Analysis: House Size vs Price")
plt.xlabel("Area (sqft)")
plt.ylabel("Price")
plt.tight_layout()
plt.show()

correlation = df["area_sqft"].corr(df["price"])
print("Correlation between area and price:", round(correlation, 3))

X = df[["area_sqft"]]
y = df["price"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f"Regression equation: Price = {model.intercept_:.2f} + {model.coef_[0]:.2f} * Area")
print("R-squared on test data:", round(r2_score(y_test, y_pred), 3))
print("RMSE on test data:", round(mean_squared_error(y_test, y_pred) ** 0.5, 2))
```

Output

```
Correlation between area and price: 0.595
Regression equation: Price = 469664.72 + 120.87 * Area
R-squared on test data: 0.205
RMSE on test data: 189920.13
```



Experiment 33

Linear Regression for Car Price Prediction

PYTHON

Code

```
# Q33: Linear Regression for Car Price Prediction (multi-feature)
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

df = pd.read_csv("car_price_multi.csv")
X = df[["engine_size", "horsepower", "fuel_efficiency_kmpl"]]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Model coefficients:")
for feature, coef in zip(X.columns, model.coef_):
    print(f"  {feature}: {coef:.2f}")
print("Intercept:", round(model.intercept_, 2))
print("R-squared on test data:", round(r2_score(y_test, y_pred), 3))

most_influential = X.columns[abs(model.coef_).argmax()]
```

```
print(f"\nMost influential factor on price: {most_influential}")
```

Output

```
Model coefficients:
  engine_size: 2491.14
  horsepower: 79.65
  fuel_efficiency_kmpl: -122.75
Intercept: 4607.88
R-squared on test data: 0.937

Most influential factor on price: engine_size
```

Experiment 34

KNN for Treatment Outcome Prediction

PYTHON

Code

```
# Q34: KNN for Treatment Outcome Prediction
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

df = pd.read_csv("treatment.csv")

le_gender = LabelEncoder()
df["gender_enc"] = le_gender.fit_transform(df["gender"])

le_outcome = LabelEncoder()
df["outcome_enc"] = le_outcome.fit_transform(df["outcome"])  # Bad=0, Good=1

X = df[["age", "gender_enc", "blood_pressure", "cholesterol"]]
y = df["outcome_enc"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)

print("Model Evaluation on Test Set:")
print("Accuracy: ", round(accuracy_score(y_test, y_pred), 3))
print("Precision:", round(precision_score(y_test, y_pred), 3))
print("Recall:   ", round(recall_score(y_test, y_pred), 3))
print("F1-Score: ", round(f1_score(y_test, y_pred), 3))

print("\nPredictions on test set (first 10):")
results = pd.DataFrame({
    "Actual": le_outcome.inverse_transform(y_test[:10]),
    "Predicted": le_outcome.inverse_transform(y_pred[:10])
})
print(results.to_string(index=False))
```

Output

```
Model Evaluation on Test Set:
```

```
Accuracy:  0.633
Precision: 0.6
Recall:    0.8
F1-Score:  0.686

Predictions on test set (first 10):
Actual Predicted
   Bad       Bad
   Bad       Bad
  Good      Good
   Bad       Bad
  Good      Good
   Bad      Good
   Bad       Bad
  Good       Bad
   Bad      Good
   Bad       Bad
```

Experiment 35

K-Means for Retail Customer Segmentation

PYTHON

Code

```
# Q35: K-Means for Retail Customer Segmentation
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("retail_customers.csv")
X = df[["total_spent", "visit_frequency"]]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = KMeans(n_clusters=3, n_init=10, random_state=42)
df["segment"] = model.fit_predict(X_scaled)

print("Segment sizes:")
print(df["segment"].value_counts().sort_index())

print("\nSegment profile (mean values):")
print(df.groupby("segment")[["total_spent", "visit_frequency"]].mean().round(2))
```

Output

```
Segment sizes:
segment
0    50
1    48
2    52
Name: count, dtype: int64

Segment profile (mean values):
         total_spent  visit_frequency
segment
0            7427.24            68.08
1            4930.70            16.12
2            2264.00            63.62
```

Experiment 36

Stock Price Variability Analysis

PYTHON

Code

```
# Q36: Stock Price Variability Analysis
import pandas as pd
import numpy as np

df = pd.read_csv("stock_prices.csv")
prices = df["closing_price"]

mean_price = prices.mean()
std_price = prices.std()
min_price = prices.min()
max_price = prices.max()
price_range = max_price - min_price
cv = (std_price / mean_price) * 100

daily_returns = prices.pct_change().dropna()

print("Stock Price Variability Analysis")
print("Mean closing price:", round(mean_price, 2))
print("Standard deviation:", round(std_price, 2))
print("Minimum price:", round(min_price, 2))
print("Maximum price:", round(max_price, 2))
print("Price range:", round(price_range, 2))
print("Coefficient of variation (%):", round(cv, 2))
print("Average daily return (%):", round(daily_returns.mean() * 100, 3))
print("Daily return volatility (std, %):", round(daily_returns.std() * 100, 3))

if cv < 10:
    print("\nInsight: The stock shows low variability - relatively stable price movement.")
elif cv < 25:
    print("\nInsight: The stock shows moderate variability.")
else:
    print("\nInsight: The stock shows high variability - a relatively volatile price history.")
```

Output

```
Stock Price Variability Analysis
Mean closing price: 517.64
Standard deviation: 10.6
Minimum price: 497.93
Maximum price: 552.84
Price range: 54.91
Coefficient of variation (%): 2.05
Average daily return (%): 0.091
Daily return volatility (std, %): 1.005

Insight: The stock shows low variability - relatively stable price movement.
```

Experiment 37

Correlation Between Study Time and Exam Scores

PYTHON

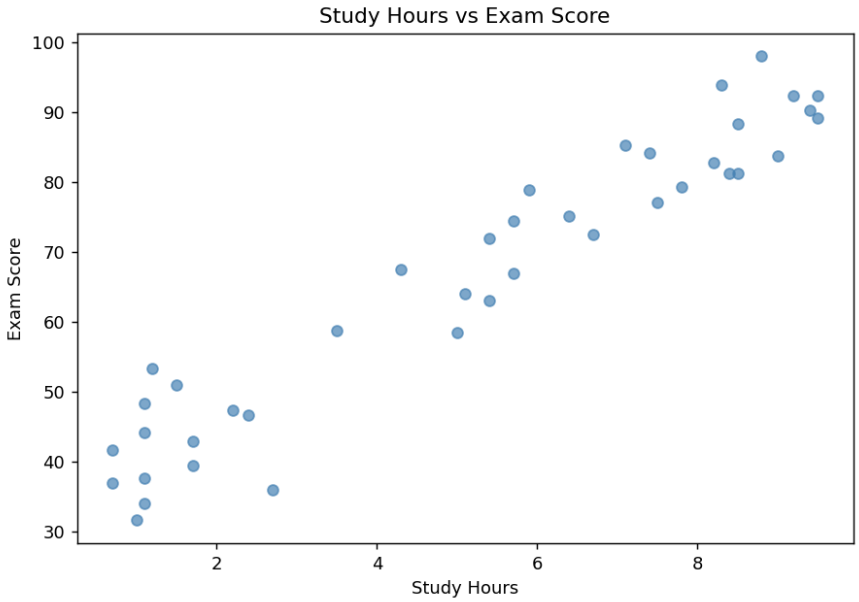
Code

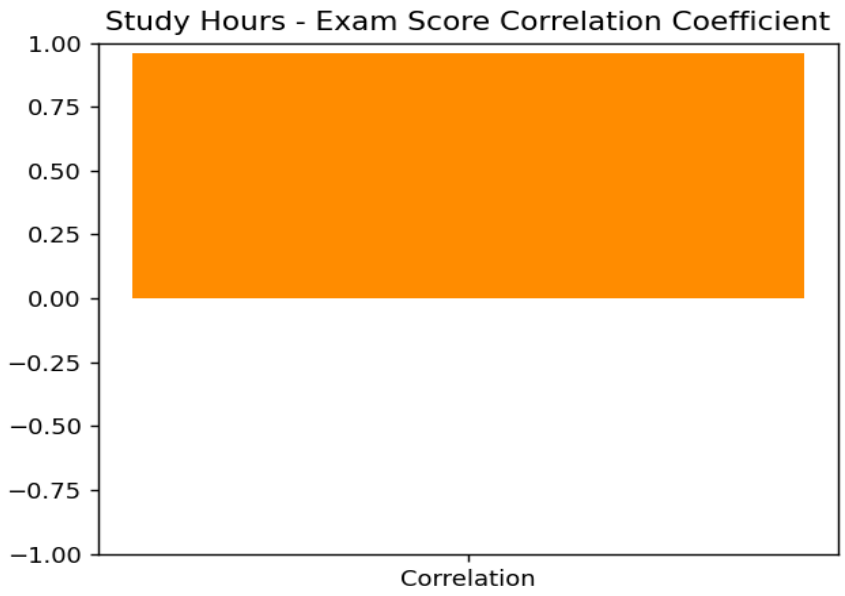
```
# Q37: Correlation Between Study Time and Exam Scores
```

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("study_scores.csv")

correlation = df["study_hours"].corr(df["exam_score"])
print("Correlation between study hours and exam scores:", round(correlation, 3))

if correlation > 0.7:
    print("Insight: Strong positive correlation - more study time is strongly associated with higher scor
    es.")
elif correlation > 0.3:
    print("Insight: Moderate positive correlation.")
else:
    print("Insight: Weak or no correlation.")

# Scatter plot with trend line
plt.figure(figsize=(7, 5))
plt.scatter(df["study_hours"], df["exam_score"], color="steelblue", alpha=0.7)
plt.title("Study Hours vs Exam Score")
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.tight_layout()
plt.show()

# Correlation heatmap-style bar (simple visualization)
plt.figure(figsize=(5, 4))
plt.bar(["Correlation"], [correlation], color="darkorange")
plt.ylim(-1, 1)
plt.title("Study Hours - Exam Score Correlation Coefficient")
plt.tight_layout()
plt.show()
```

Output

```
Correlation between study hours and exam scores: 0.959
Insight: Strong positive correlation - more study time is strongly associated with higher scores.
```





Experiment 38

Weather Data Variability Analysis

PYTHON

Code

```
# Q38: Weather Data Variability Analysis
import pandas as pd

df = pd.read_csv("weather_data.csv")

summary = df.groupby("city")["temperature"].agg(
    mean_temp="mean", std_temp="std", min_temp="min", max_temp="max"
)
summary["temp_range"] = summary["max_temp"] - summary["min_temp"]
summary = summary.round(2)

print("City-wise Temperature Summary:")
print(summary)

highest_range_city = summary["temp_range"].idxmax()
most_consistent_city = summary["std_temp"].idxmin()

print(f"\nCity with the highest temperature range: {highest_range_city} ({summary.loc[highest_range_city,
     'temp_range']} degrees C)")
print(f"Most consistent city (lowest std dev): {most_consistent_city} ({summary.loc[most_consistent_city,
     'std_temp']} degrees C)")
```

Output

```
City-wise Temperature Summary:
           mean_temp  std_temp  min_temp  max_temp  temp_range
city
Bengaluru      22.96      5.07       9.3      37.3        28.0
Chennai        29.62      4.07      20.5      42.1        21.6
Delhi          24.93      9.55       3.4      49.7        46.3
Mumbai         27.97      3.20      18.6      36.2        17.6
City with the highest temperature range: Delhi (46.3 degrees C)
```

```
Most consistent city (lowest std dev): Mumbai (3.2 degrees C)
```

Experiment 39

K-Means Clustering for Purchase Behavior

PYTHON

Code

```
# Q39: K-Means Clustering for Purchase Behavior + Visualization
import warnings; warnings.filterwarnings("ignore")
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("purchase_behavior.csv")
X = df[["total_amount", "items_purchased"]]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = KMeans(n_clusters=4, n_init=10, random_state=42)
df["cluster"] = model.fit_predict(X_scaled)

print("Cluster sizes:")
print(df["cluster"].value_counts().sort_index())
print("\nCluster profile (mean values):")
print(df.groupby("cluster")[["total_amount", "items_purchased"]].mean().round(2))

plt.figure(figsize=(7, 5))
scatter = plt.scatter(df["total_amount"], df["items_purchased"], c=df["cluster"], cmap="viridis", s=45)
plt.title("Customer Clusters: Total Amount vs Items Purchased")
plt.xlabel("Total Amount Spent")
plt.ylabel("Items Purchased")
plt.colorbar(scatter, label="Cluster")
plt.tight_layout()
plt.show()
```

Output

```
Cluster sizes:
cluster
0    46
1    39
2    32
3    33
Name: count, dtype: int64

Cluster profile (mean values):
         total_amount  items_purchased
cluster
0             4484.83            36.72
1            10173.67            12.56
2            12115.45            38.72
3             3197.78            15.18
```

Experiment 40

Soccer Player Analysis

PYTHON

Code

```
# Q40: Soccer Player Analysis
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("soccer_players.csv")

print("Top 5 players by goals scored:")
print(df.nlargest(5, "goals")[["name", "goals"]].to_string(index=False))

print("\nTop 5 players by salary:")
print(df.nlargest(5, "salary")[["name", "salary"]].to_string(index=False))

avg_age = df["age"].mean()
print(f"\nAverage age of players: {avg_age:.1f}")

above_avg = df[df["age"] > avg_age]
print(f"\nPlayers above average age ({len(above_avg)} players):")
print(above_avg[["name", "age"]].to_string(index=False))

position_counts = df["position"].value_counts()
plt.figure(figsize=(7, 5))
plt.bar(position_counts.index, position_counts.values, color="teal")
plt.title("Distribution of Players by Position")
plt.xlabel("Position")
plt.ylabel("Number of Players")
plt.tight_layout()
plt.show()
```

Output

```
Top 5 players by goals scored:
         name  goals
```

```
 Leke Adebayo     28
Miguel Torres     26
Diego Fuentes     24
 Andres Villa     22
  Yusuf Demir     21

Top 5 players by salary:
         name  salary
 Leke Adebayo  185000
Miguel Torres  175000
Diego Fuentes  160000
 Andres Villa  145000
  Samuel Osei  130000

Average age of players: 26.7

Players above average age (10 players):
          name  age
  Leke Adebayo   27
    Tomas Kral   30
 Diego Fuentes   29
Rashid Karimov   31
   Samuel Osei   28
Nikolai Petrov   33
  Andres Villa   27
    Ivan Kovac   29
    Jonas Berg   32
 Miguel Torres   30
```